

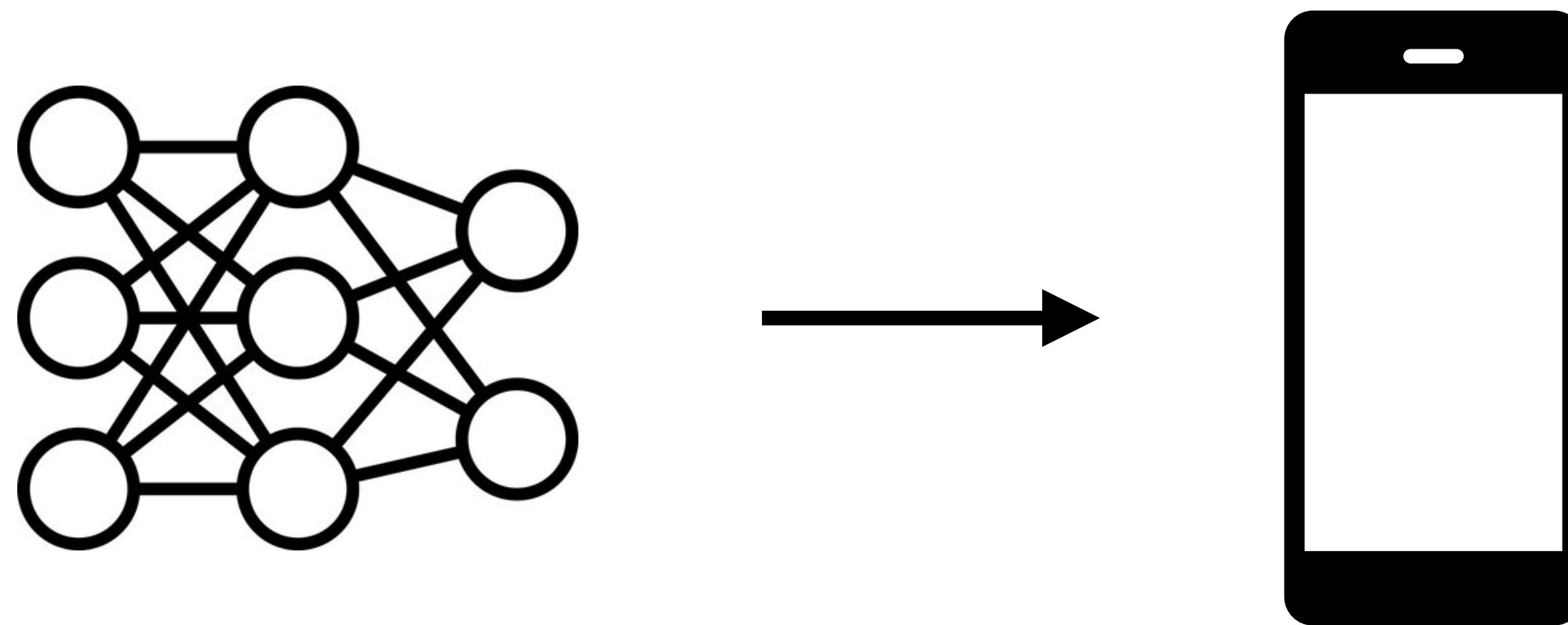
Сжатие и ускорение моделей

План

- Дистилляция
- Квантизация
- Спекулятивный декодинг

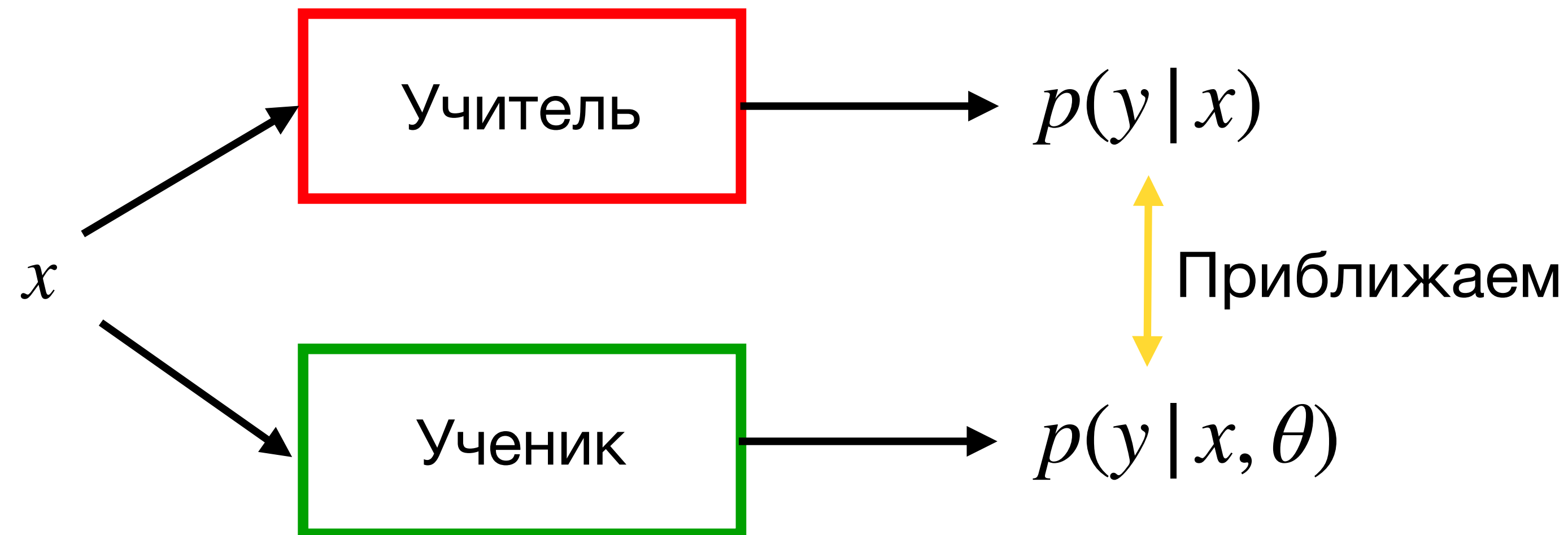
Мотивация

- С каждым годом нейронные сети становятся больше
- При этом часто хочется использовать модели локально на портативных устройствах
- Для этого нужно научиться уменьшать их размеры без потери качества



Дистилляция

- Дистилляция знаний – процесс переноса знаний из большой обученной модели (учителя) в маленькую модель (ученика) с минимальной потерей качества
- Дистилляция реализуется путем приближения выходов ученика к выходам учителя



Как именно приближать?

Для приближения вероятностей минимизируется одна из двух ошибок:

- **KL-дивергенция**

$$KL(p^s \| p^t) = \sum_{k=1}^K p_k^s \log \frac{p_k^s}{p_k^t}$$

- **Кросс-энтропия**

$$CE(p^t, p^s) = - \sum_{k=1}^K p_k^t \log p_k^s$$

Как именно приближать?

Для приближения вероятностей минимизируется одна из двух ошибок:

- **KL-дивергенция**

$$KL(p^s \| p^t) = \sum_{k=1}^K p_k^s \log \frac{p_k^s}{p_k^t}$$

- **Кросс-энтропия**

$$\begin{aligned} CE(p^t, p^s) &= - \sum_{k=1}^K p_k^t \log p_k^s \propto \\ &\propto \sum_{k=1}^K p_k^t \log p_k^t - \sum_{k=1}^K p_k^t \log p_k^s = KL(p^t \| p^s) \end{aligned}$$

Сглаживание вероятностей

- Нейронные сети очень часто бывают слишком уверены в предсказаниях
- Распределение p^t оказывается почти вырожденным
- Приближение выходов к такому распределению не лучше обычного обучения с учителем

В дистилляции используются **сглаженные** вероятности.

$$\begin{aligned} p_{\tau}^t &= \text{softmax}(l^t, T = \tau) \\ p_{\tau}^s &= \text{softmax}(l^s, T = \tau) \end{aligned} \quad \tau > 1$$

Такие метки называются **мягкими**.

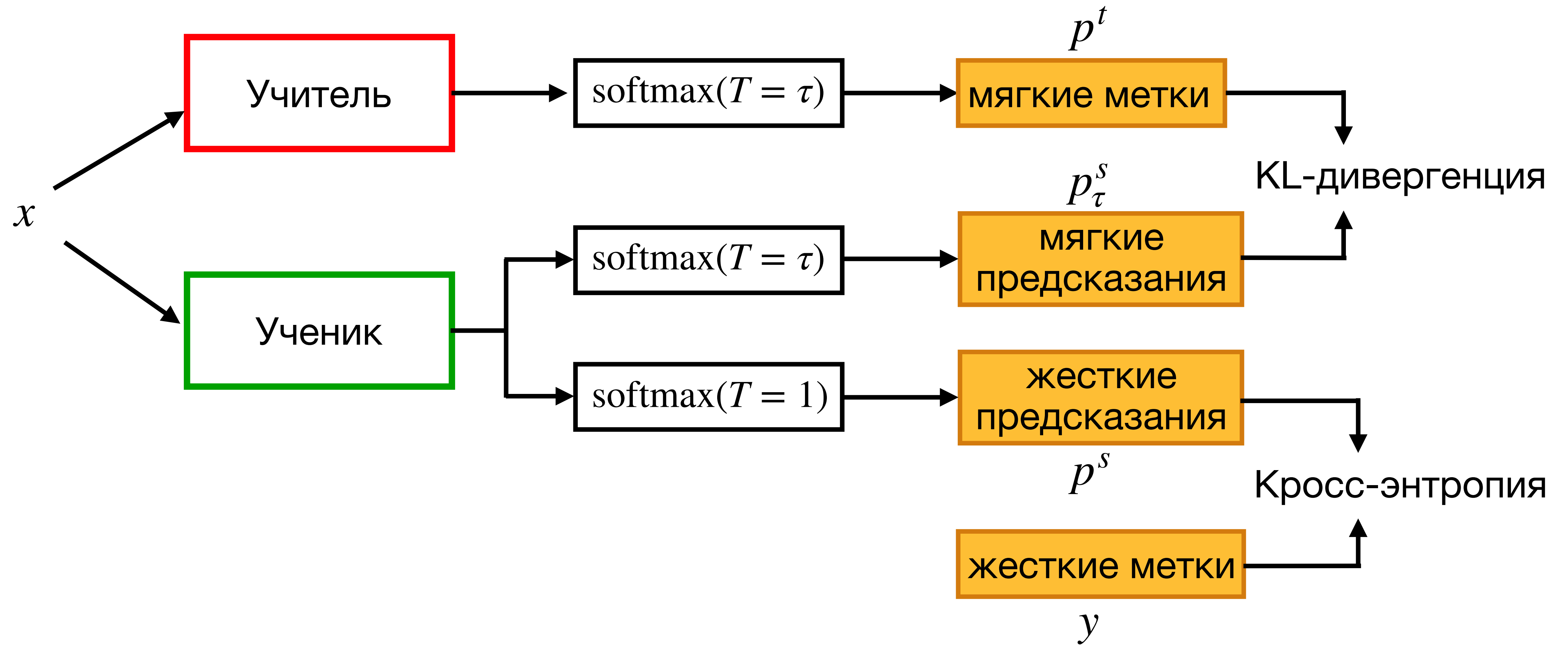
Жесткие метки

Так как у нас есть доступ к правильным ответам, добавляем так же стандартную ошибку.

$$CE(y, p^s) = - \sum_{k=1}^K [y = k] \log p_k^s$$

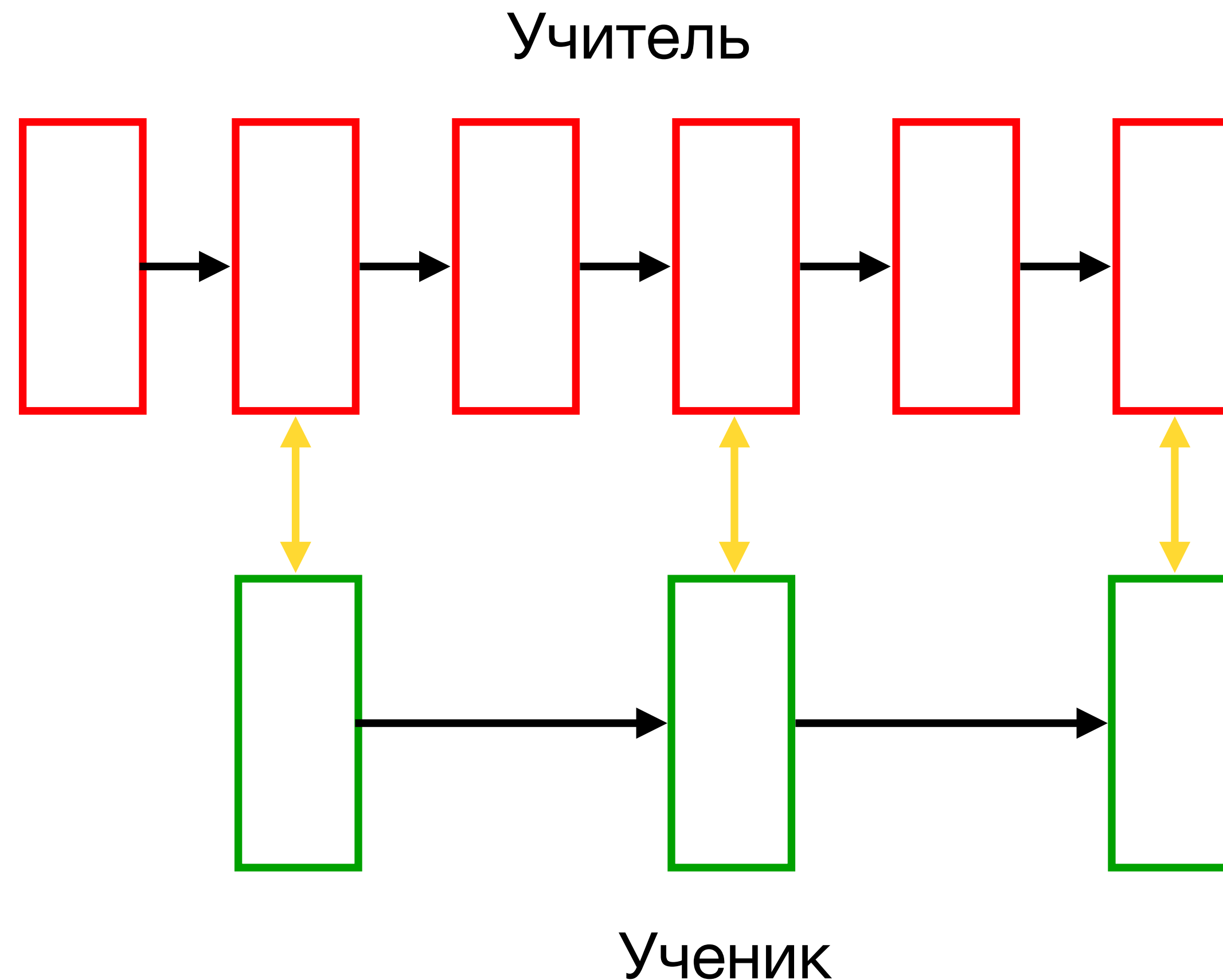
Правильные ответы y называются **жесткими** метками.

Схема обучения



Приближение промежуточных слоев

Мы приближаем только выходы моделей, но есть же еще выходы слоев!
Будем выходы слоев ученика приближать к выходам учителя



Приближение слоев внимания

Матрицы внимания имеют одинаковый размер: $[H, \text{seq_len}, \text{seq_len}]$

Введем ошибку для минимизации расстояния между ними

$$L_{attn} = \frac{1}{H} \sum_{h=1}^H \|A_h^s - A_h^t\|_2^2$$

Приближение выходов слоев

Размер выхода слоя может изменяться у разных моделей, поэтому добавим проекцию $W \in \mathbb{R}^{[d_s \times d_t]}$.

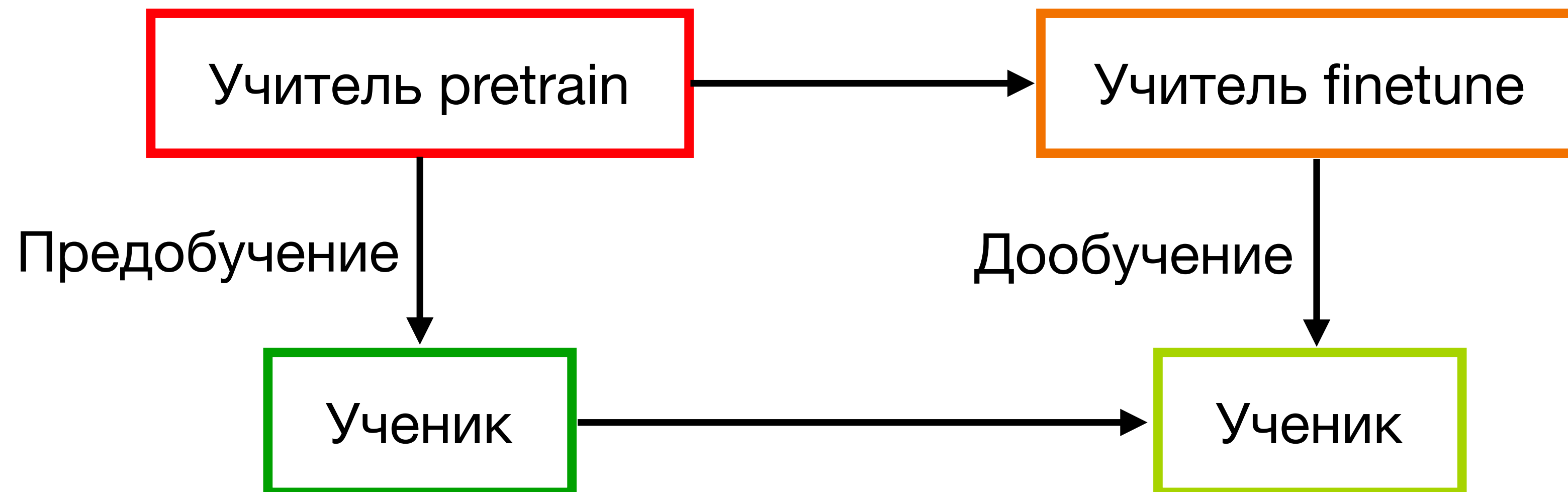
$$L_{hidn} = \|H^s W - H^t\|_2^2$$

Матрица W обучается вместе с параметрами ученика.

Дистилляция с задачей предобучения

Почти все трансформерные модели обучаются в два этапа: предобучение и дообучение.

В таком случае лучше сначала дистиллировать ученика на задаче предобучения, а затем на задаче дообучения.



Ограничения дистилляции

- Требуется доступ к данным
- Занимает много времени
- Большие затраты по памяти из-за необходимости хранения двух моделей

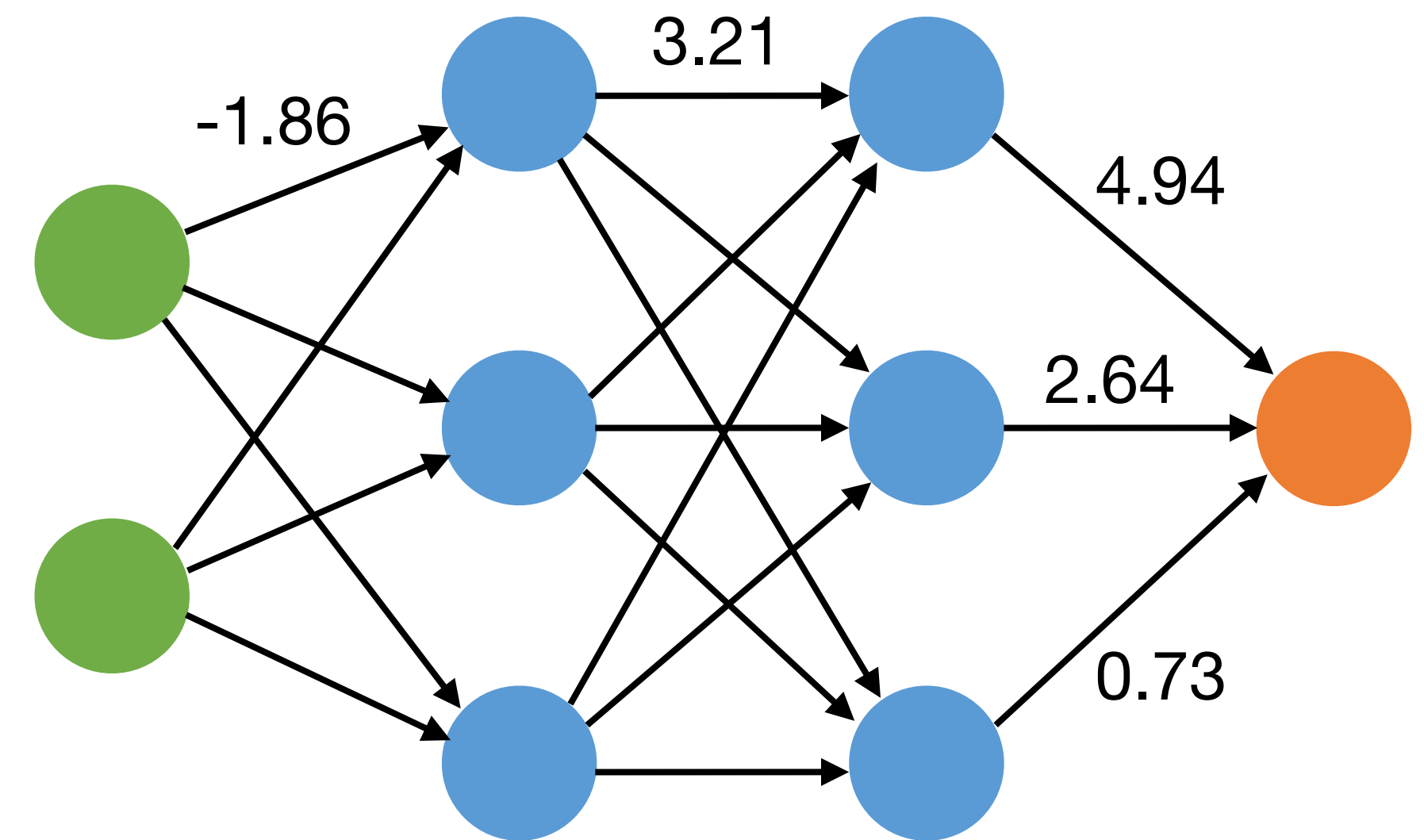
Квантизация

Квантизация

- По умолчанию нейронные сети работают с числами в формате float32.
- Это вещественные числа, которые занимают 32 бита.
- Чем больше бит используется для представления числа, тем больше нужно
 - Времени на чтение/запись
 - Памяти на хранение
 - Времени на операции сложения и умножения

Квантизация – уменьшение точности (битности) чисел.

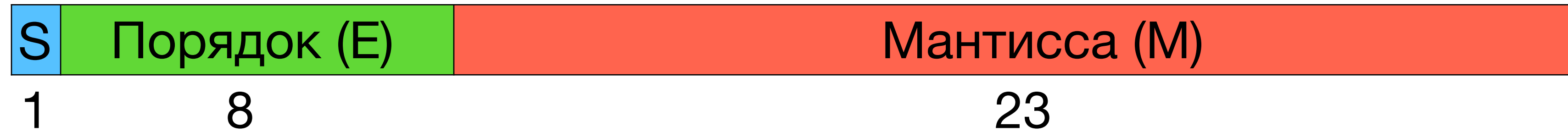
Чаще всего используется **после** обучения.



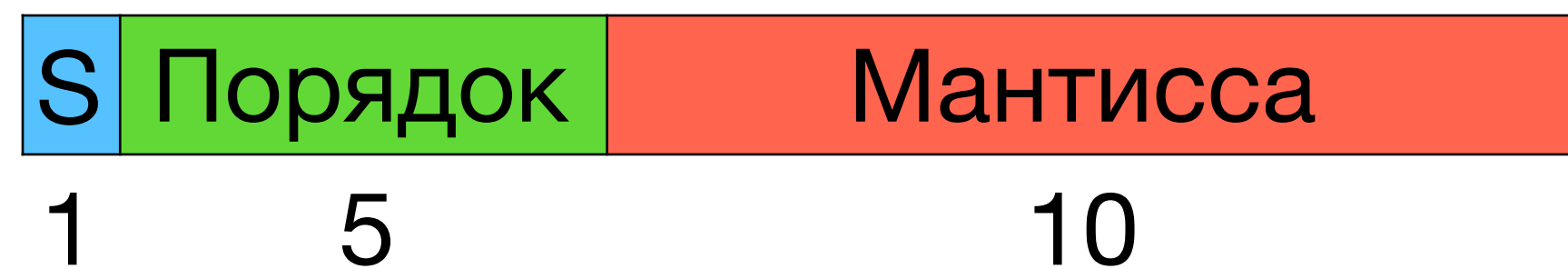
Как машины видят числа?

float32 (single-precision)

знак

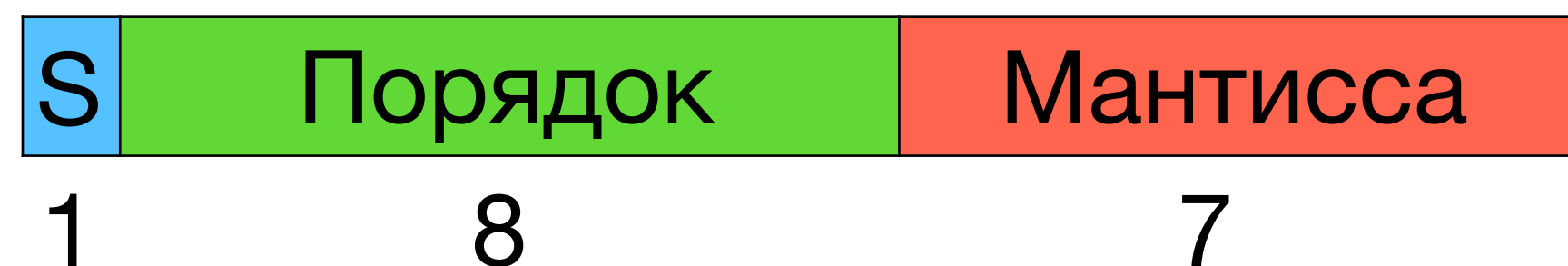


float16 (half-precision)



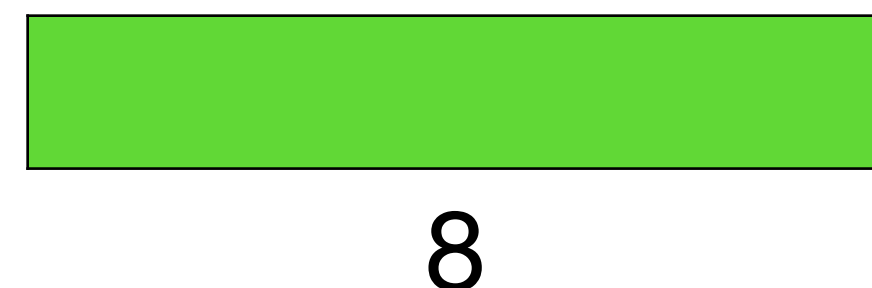
$$x = (-1)^S \cdot (1, M) \cdot 2^{E-127}$$

bf16 (brain float)



Для представления больших чисел

int8



Квантизация

Чаще всего точность понижается с **float32** до

- **float16**
- **int8** (float8 не используется, потому что он почти нигде не определен)

Квантизация в **float16** выполняется в лоб обрезанием мантиссы до нужного размера

Квантизация в **int8** работает намного хитрее

float32 -> int8

Пусть x – число в float32, x_q – квантизованное x в int8

$$x_q = \text{round}\left(\frac{x}{s} + z\right) \quad x = s \cdot (x_q - z)$$

- s – масштабирующий коэффициент; положительное число в float32
- z – сдвиг; он хранится в int8 и соответствует 0 в исходном float32 числе

Такая квантизация называется **асимметричной** и используется, когда $x_q \in [0, 255]$

Как найти s и z ?

Если мы знаем диапазон $[a, b]$ значений x и $x_q \in [a_q, b_q]$, то

$$a = s(a_q - z) \quad b = s(b_q - z)$$

Тогда

$$s = \frac{b - a}{b_q - a_q} \quad z = \frac{ba_q - ab_q}{b - a}$$

Как найти s и z ?

Если мы знаем диапазон $[a, b]$ значений x и $x_q \in [a_q, b_q]$, то

$$a = s(a_q - z) \quad b = s(b_q - z)$$

Тогда

$$s = \frac{b - a}{b_q - a_q} \quad z = \frac{ba_q - ab_q}{b - a}$$

Важно убедиться, что 0 переводится без погрешностей. То есть что z – целое число. Поэтому

$$z = \text{round} \left(\frac{ba_q - ab_q}{b - a} \right)$$

Как найти s и z ?

Если мы знаем диапазон $[a, b]$ значений x и $x_q \in [a_q, b_q]$, то

$$a = s(a_q - z) \quad b = s(b_q - z)$$

Тогда

$$s = \frac{b - a}{b_q - a_q} \quad z = \frac{ba_q - ab_q}{b - a}$$

Важно убедиться, что 0 переводится без погрешностей. То есть что z – целое число. Поэтому

$$z = \text{round} \left(\frac{ba_q - ab_q}{b - a} \right)$$

Если попадаете $x \notin [a, b]$, то округляем его до ближайшей границы.

$$x_q = \text{clip} \left(\text{round} \left(\frac{x}{s} + z \right), a_q, b_q \right)$$

Симметричная квантизация

Используется, если границы симметричны, $x \in [-a, a]$.

Для квантизации берется $x_q \in [-127, 127]$.

Так мы теряем одно значение квантизованной переменной, но избавляемся от сдвига

$$x_q = \text{round}(s \cdot x)$$

Как найти границы a и b ?

Квантизовывать нужно веса модели и входные значения (активации).

- Для весов легко – они не меняются
- Для активаций есть три способа:
 - Динамическая квантизация
 - Статичная квантизация
 - Обучение с учетом квантизации

Как найти границы a и b ?

Квантизовывать нужно веса модели и входные значения (активации).

- Для весов легко – они не меняются
- Для активаций есть три способа:
 - Динамическая квантизация – считаем границы на лету для каждой активации
 - Статичная квантизация – заранее оцениваем границы порогоня n примеров
 - Обучение с учетом квантизации – используем квантизацию во время обучения и находим границы по тренировочным примерам

Как найти границы a и b ?

Квантизовывать нужно веса модели и входные значения (активации).

- Для весов легко – они не меняются
- Для активаций есть три способа:
 - Динамическая квантизация – считаем границы на лету для каждой активации (максимальная точность, ниже скорость)
 - Статичная квантизация – заранее оцениваем границы пороگونя n примеров (ниже точность, максимальная скорость)
 - Обучение с учетом квантизации – используем квантизацию во время обучения и находим границы по тренировочным примерам (хорошая точность, максимальная скорость, модель настраивается под квантизацию)

Калибровка значений границ

Min-max: хорошо работает для весов модели

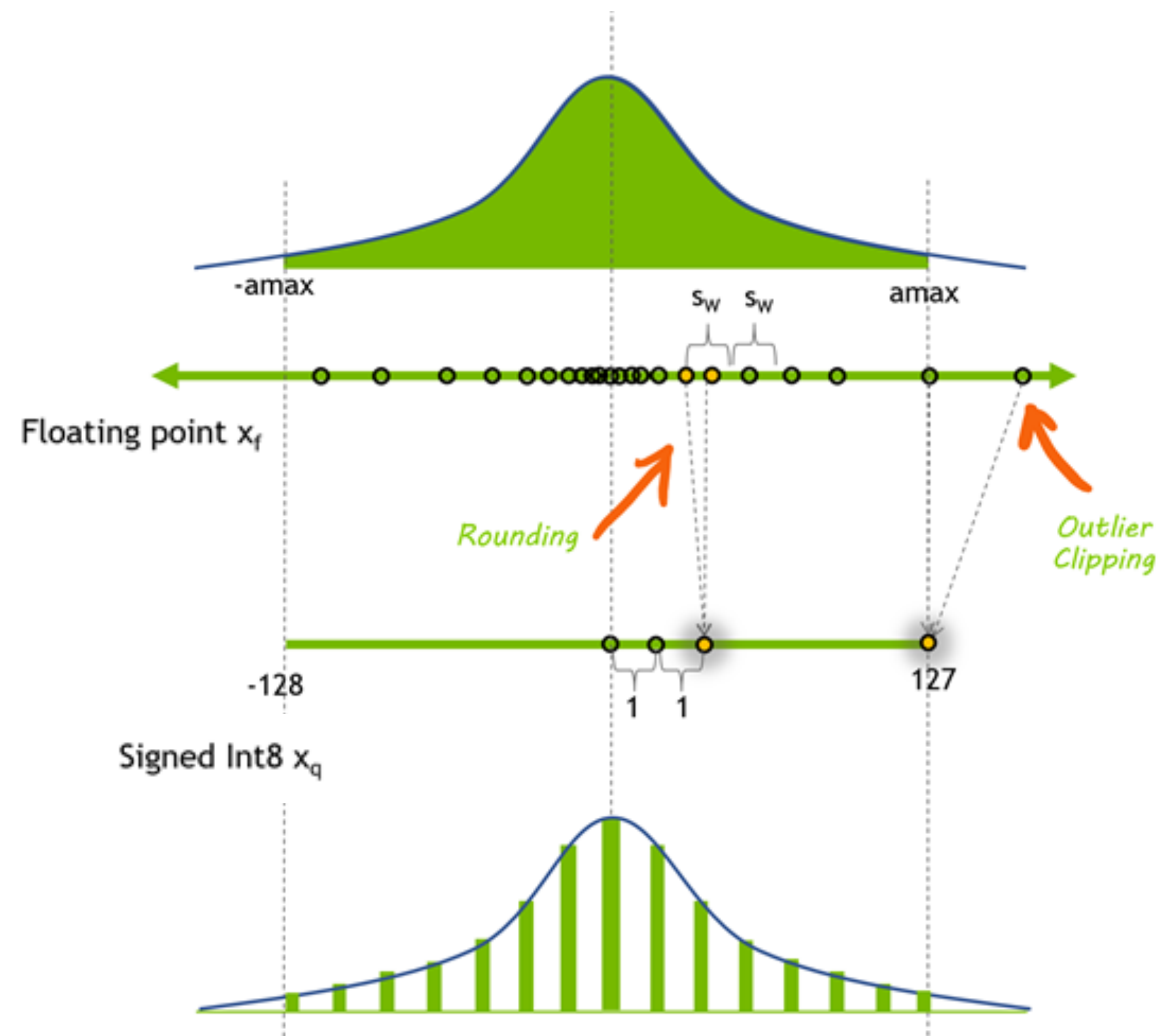
a = минимальное найденное значение

b = максимальное найденное значение

Moving average min-max: хорошо работает для активаций

a = скользящее среднее минимальное значение

b = скользящее среднее максимальное значение



Калибровка значений границ

Min-max: хорошо работает для весов модели

a = минимальное найденное значение

b = максимальное найденное значение

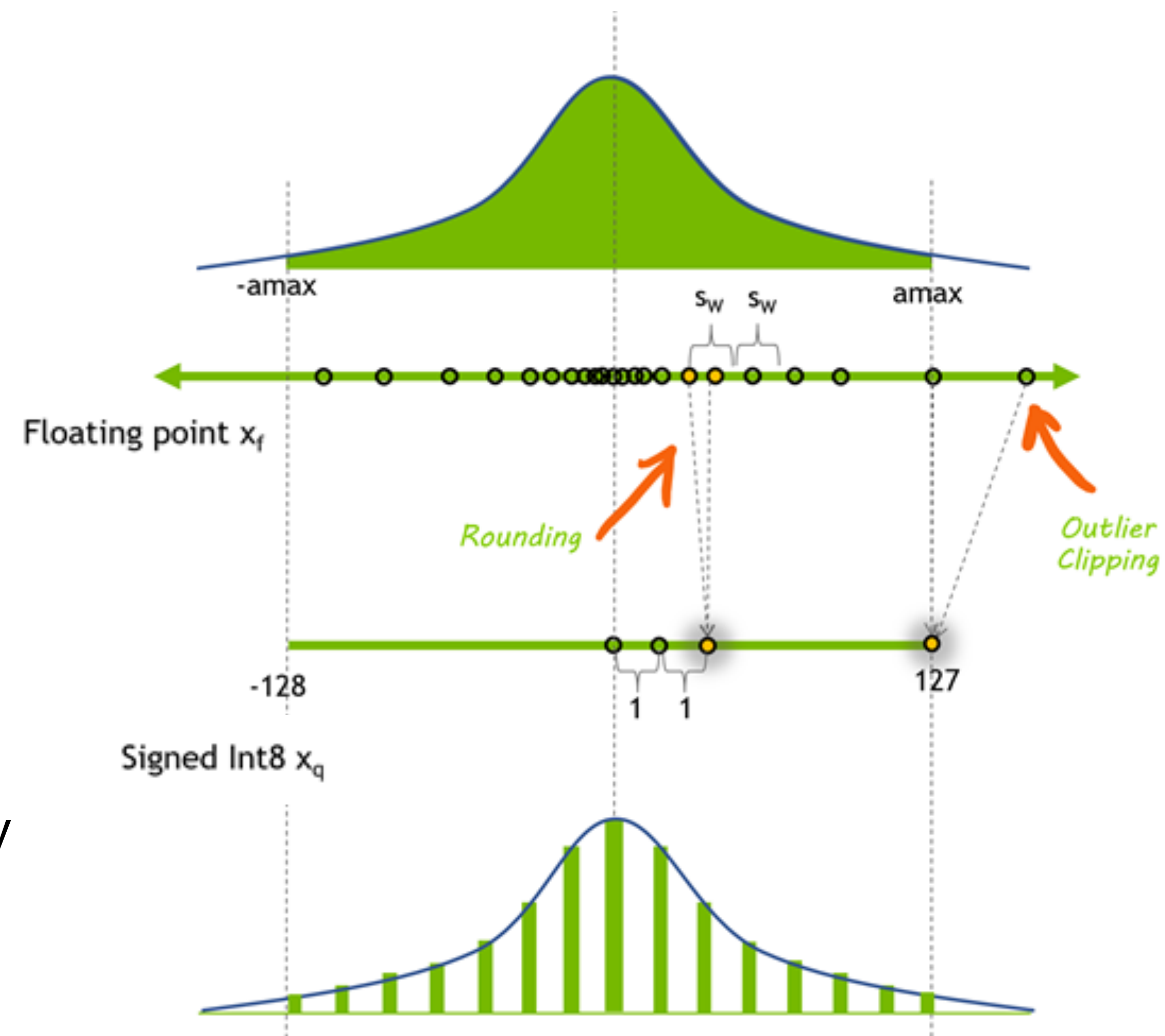
Moving average min-max: хорошо работает для активаций

a = скользящее среднее минимальное значение

b = скользящее среднее максимальное значение

Histogram-based – формируем гистограммы из всех значений активаций, находим границы одним из 3-х способов:

- **Entropy:** минимизируем KL-дивергенцию между распределениями квантизованных и исходных значений



Практические советы

- Не все операции нужно квантизовывать.
Важнее всего **матричные умножения**, так как они самые дорогие.
Перемножение двух матриц $A \in \mathbb{R}^{n \times m}$ и $B \in \mathbb{R}^{m \times p}$ требует $O(nmp)$ операций.
- Для Layer Norm лучше не применять квантизацию, так как она может работать не стабильно
- Лучше всего начать с динамической квантизации.
Если скорости не хватает, то перейти к статической.
В конце, если не хватает точности, то к обучению с учетом квантизации.

Операции с квантизованными числами

Сложение:

$$y = x_1 + x_2 = s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2)$$

Операции с квантизованными числами

Сложение:

$$y = x_1 + x_2 = s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$

Операции с квантизованными числами

Сложение:

$$y = x_1 + x_2 = s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$

$$y_q = z_y + \frac{1}{s_y} \left(s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) \right)$$

Операции с квантизованными числами

Сложение:

$$y = x_1 + x_2 = s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$
$$y_q = z_y + \frac{1}{s_y} \left(s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) \right)$$

Умножение:

$$y = x_1 \cdot x_2 = s_1(x_{q1} - z_1) \cdot s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$

Операции с квантизованными числами

Сложение:

$$y = x_1 + x_2 = s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$
$$y_q = z_y + \frac{1}{s_y} \left(s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) \right)$$

Умножение:

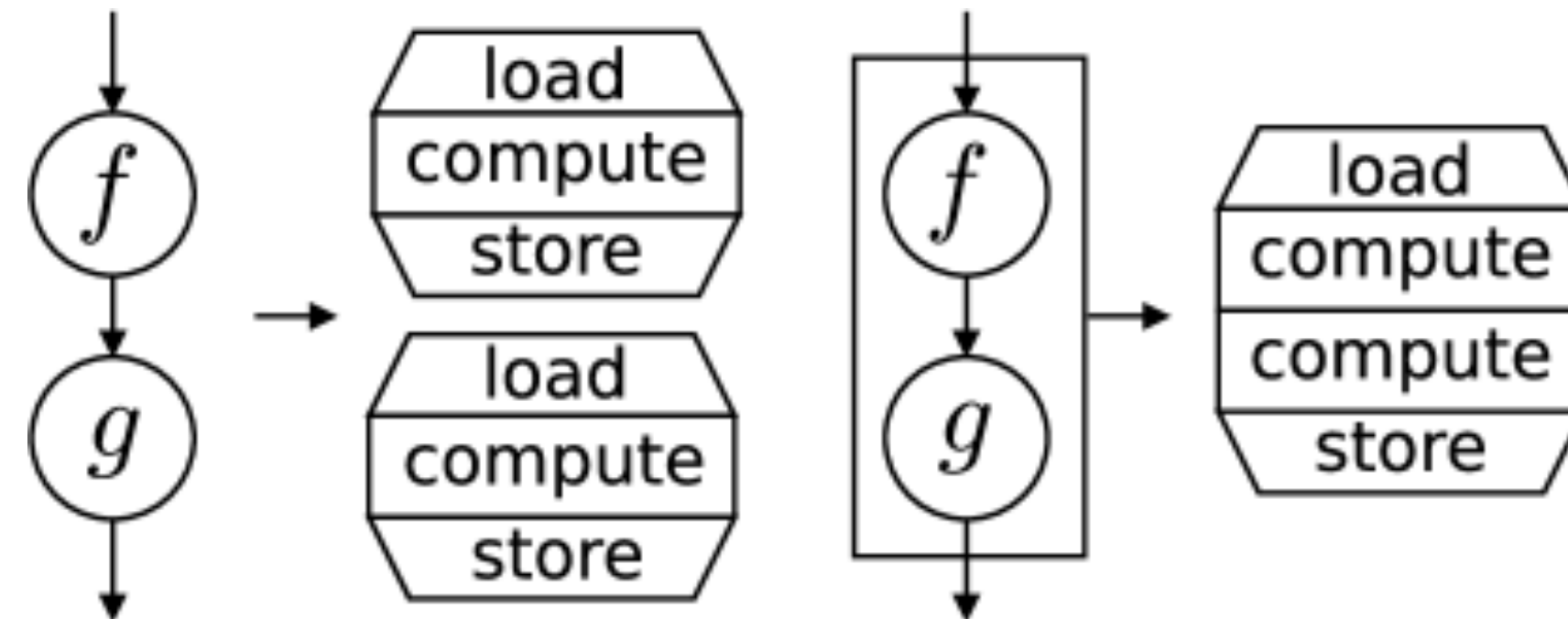
$$y = x_1 \cdot x_2 = s_1(x_{q1} - z_1) \cdot s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$
$$y_q = z_y + \frac{s_1 s_2}{s_y} (x_{q1} - z_1)(x_{q2} - z_2)$$

Почему операции с `int8` быстрее?

- Произведение в `int8` может привести к переполнению.
 - Поэтому результат записывается в `int32`. (`int8 x int8 -> int32`)
 - Тогда почему умножение в `int8` быстрее?
1. `int8` занимает меньше памяти -> чтение/запись работают быстрее
 2. Многие процессоры поддерживают ускоренное умножение `int8` матриц
 3. Перевод в `int32` осуществляется в момент записи результата

Kernel fusion

- Объединение нескольких последовательных операций в одну
- Экономит время на чтении/записи



Kernel fusion

- Объединение нескольких последовательных операций в одну
- Экономит время на чтении/записи

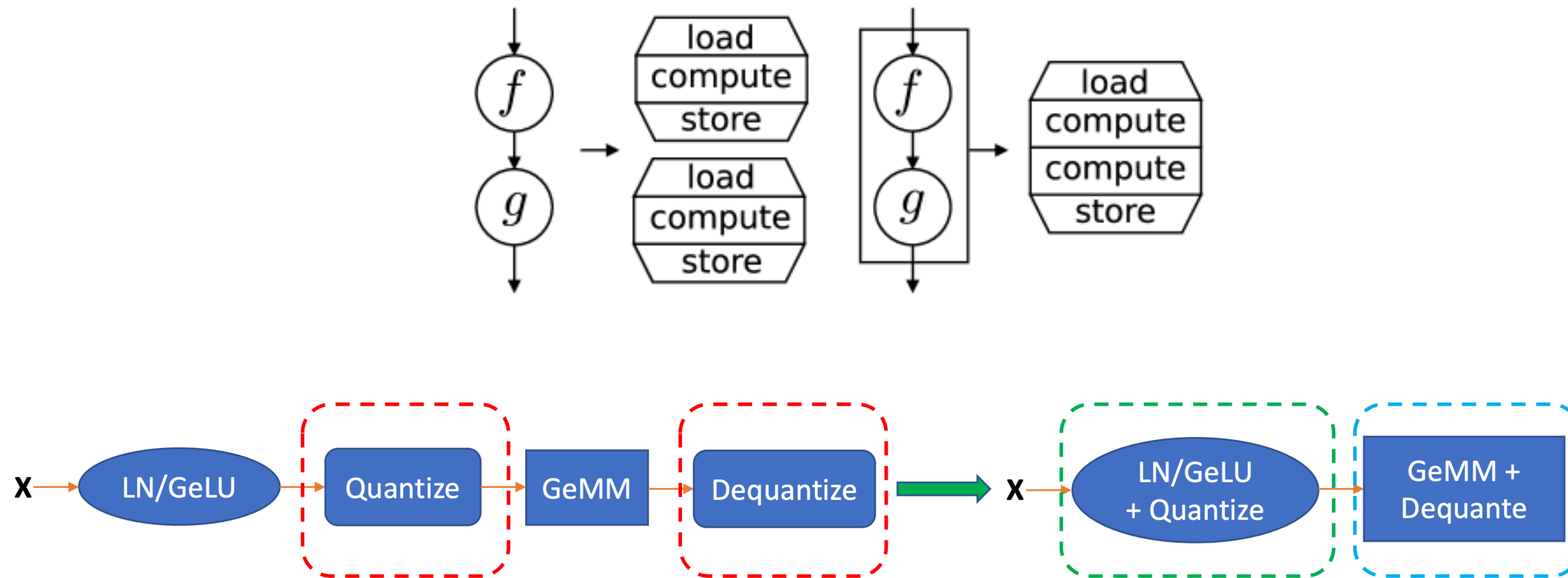


Figure 2: The illustration of normal (left) and our fused (right) INT8 GeMM.

Наивная квантизация работает плохо

Мотивация: активации и веса модели имеют довольно много выбросов, из-за которых обычная квантизация работает плохо

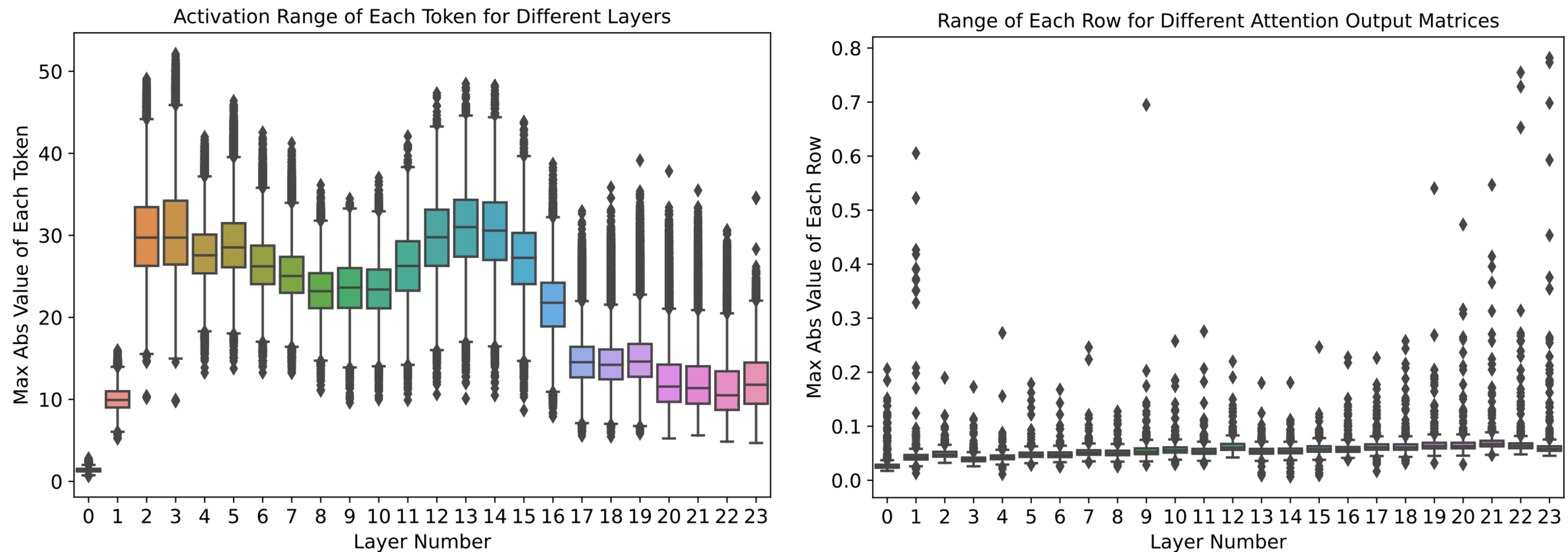


Figure 1: The activation range (left) and row-wise weight range of the attention output matrix (right) different layers on the pretrained GPT-3_{350M}. See Figure C.1 for the results of BERT_{base}.

Наивная квантизация работает плохо

Стандартная квантизация и весов, и активаций заметно портит качество.

Table 1: Post training quantization results of GPT-3_{350M} on 20 zero-shot evaluation datasets. Here WxAy means x-/y-bit for weight/activation. Particularly, for W4/8, we quantize the MHSA’s weight to INT8 and FFC’s weight to INT4. Please see Table [H.1](#) for the results of all 20 tasks.

Precision	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)
W16A16	49.3	66.3	29.4	53.8	75.1	38.9	21.5
W8A16	49.3	66.1	29.6	54.2	74.8	38.5	22.1
W16A8	44.7	64.8	28.2	52.7	69.2	37.8	24.6
W8A8	42.6	64.1	28.0	53.1	67.5	37.8	26.2
W4/8A16	0.00	51.4	30.2	52.7	16.1	28.9	1.76e5

Per-tensor и per-channel квантизация

Так как параметры квантизации занимают место, хочется минимизировать их число

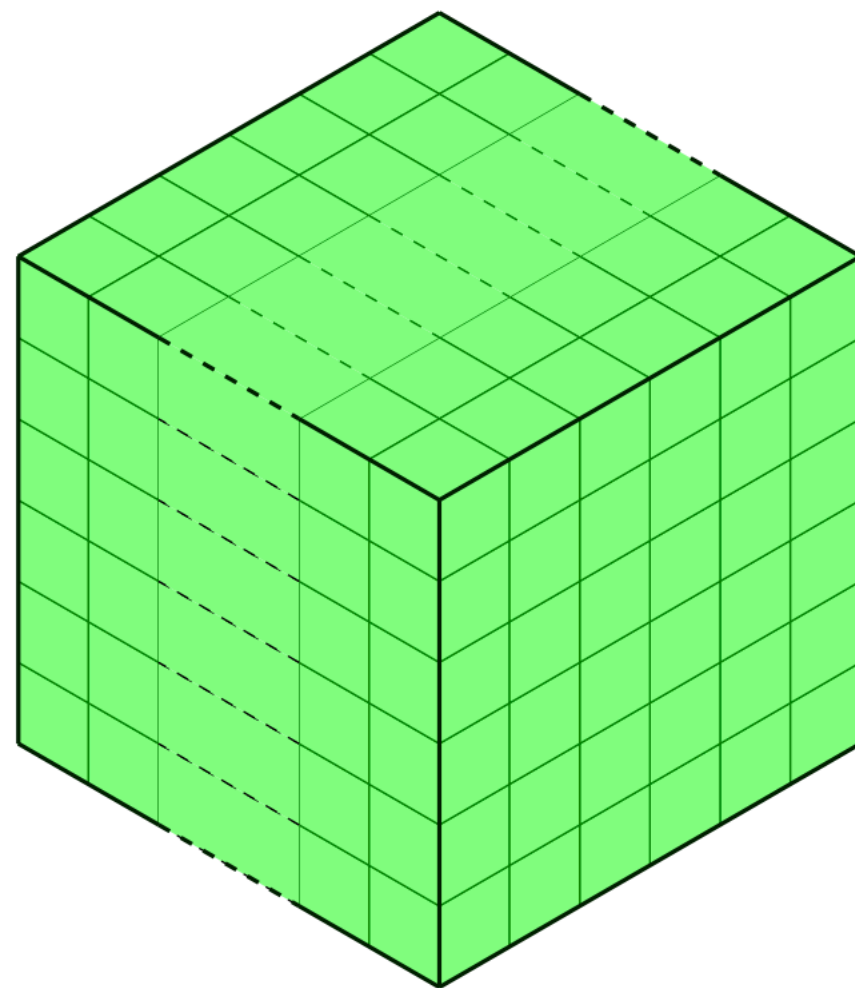
- **Per-tensor:** храним одну пару $(s; z)$ для каждого тензора
- **Per-channel:** храним по паре для каждой координаты вектора

Меньше памяти

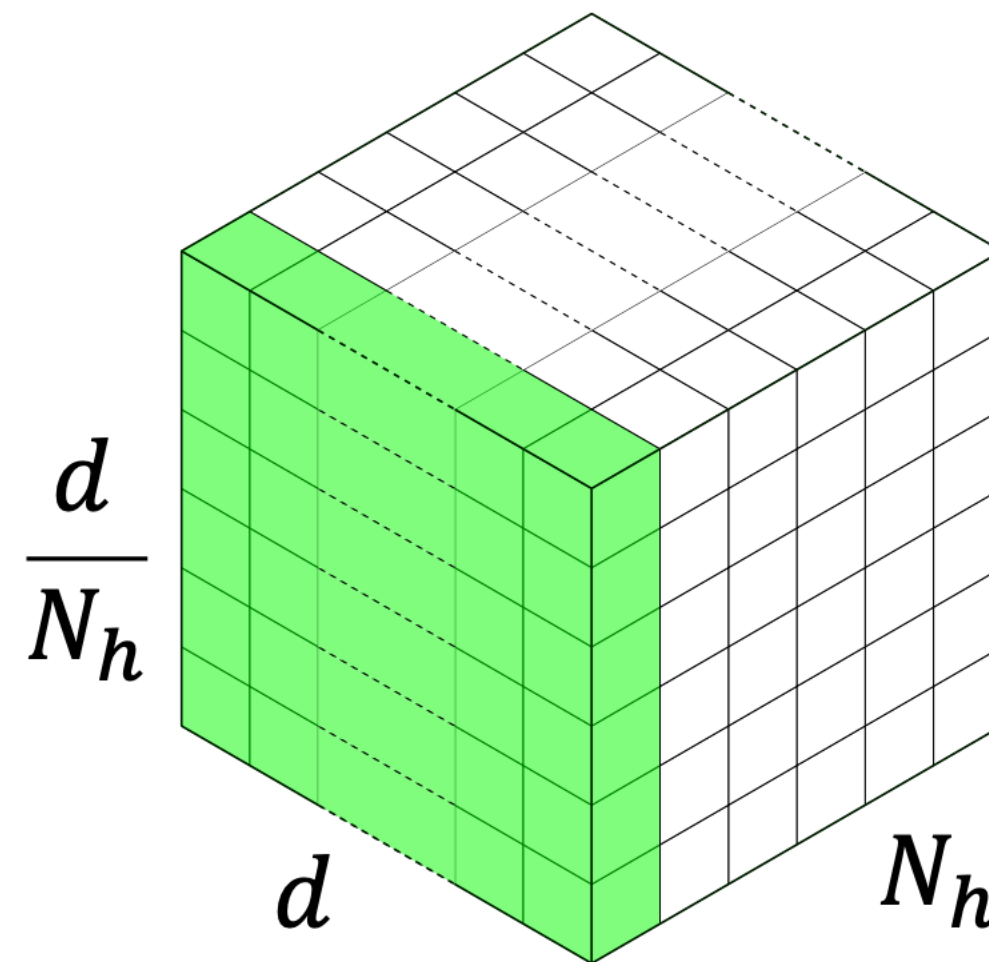
Выше точность

ZeroQuant

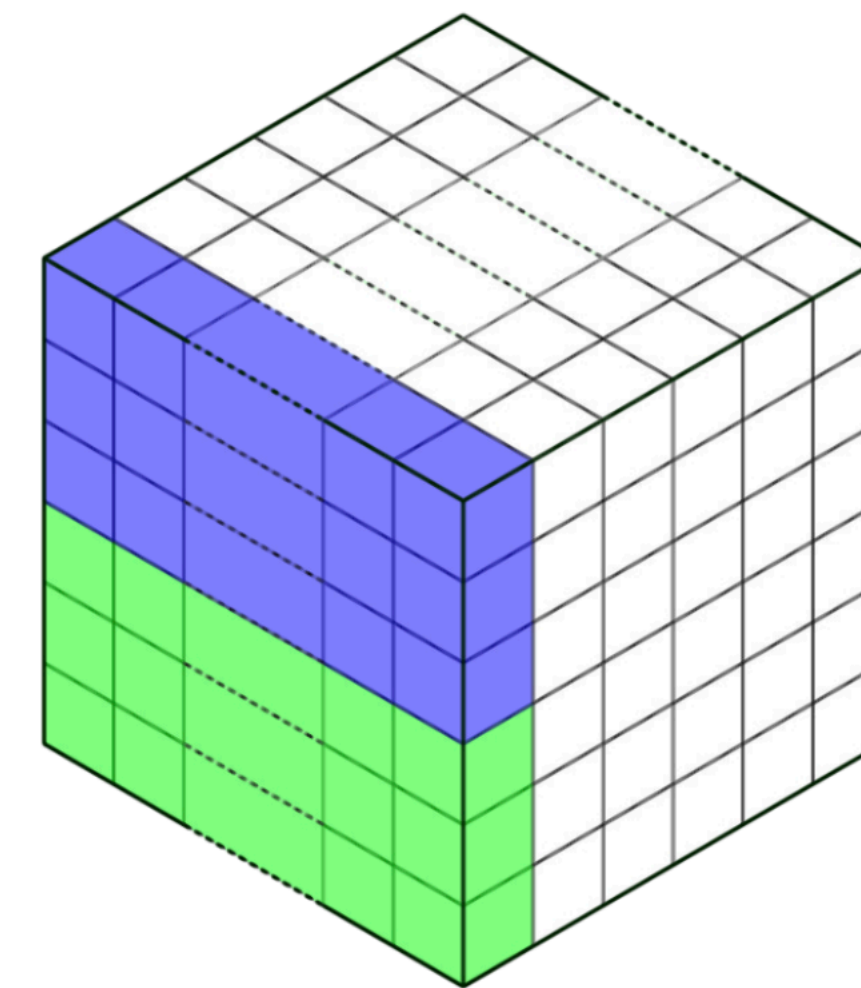
- Первая *динамическая* квантизация для LLM
- Квантизует токены (строки матриц весов) независимо от других токенов (строк)



(a) Layer-wise



(b) Group-wise (N_h group)



(c) Group-wise ($2N_h$ group)

ZeroQuant

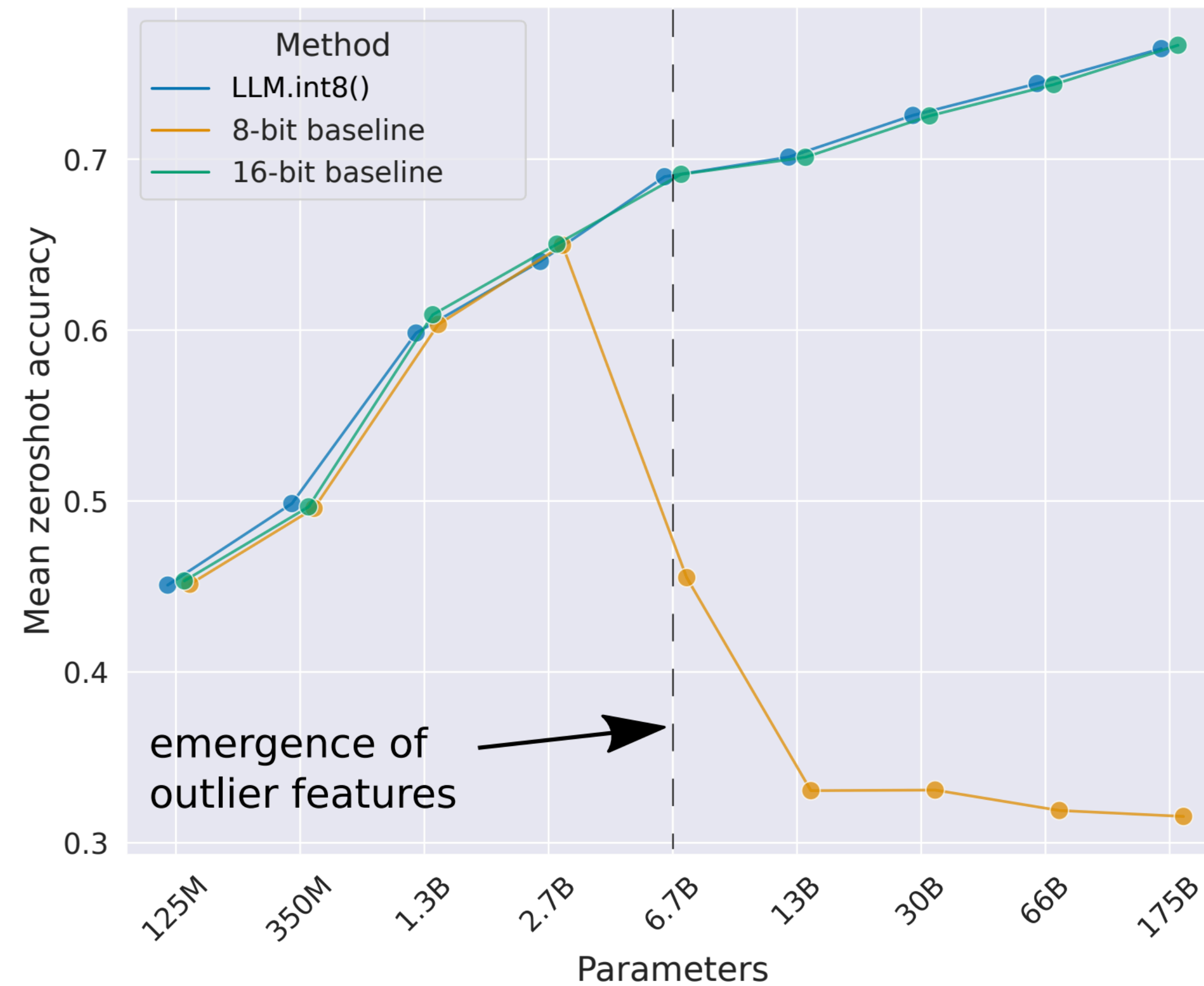
Значительно улучшает качество относительно обычной статической квантизации.
Ускорение в 2-5 раз для BERT

Precision (Method)	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)	Time Cost
W16A16	61.3	71.4	33.6	53.1	82.6	42.4	15.3	N/A
W8A8 (PTQ)	54.8	67.7	16.6	54.5	75.7	40.5	18.9	13 mins
W8A8 (ZeroQuant)	62.6	70.7	33.4	52.7	80.9	42.3	15.7	0
W4/8A16 (PTQ)	0.00	50.4	27.0	50.9	15.8	29.0	1.35e5	13 mins
W4/8A16 (ZeroQuant)	43.9	66.5	30.0	52.7	77.3	39.38	21.9	0
W4/8A16 (ZeroQuant-LKD)	59.4	69.5	31.6	52.7	79.7	41.5	17.6	3 hours
W4/8A8 (ZeroQuant)	46.8	66.4	28.8	52.7	76.2	39.24	24.1	0
W4/8A8 (ZeroQuant-LKD)	48.7	68.1	29.0	52.0	77.4	39.90	18.2	3 hours

Here WxAy means x-/y-bit for weight/activation. Particularly, for W4/8, we quantize the MHSA’s weight to INT8 and FFC’s weight to INT4. LKD is a layer-by-layer knowledge distillation

LLM.int8()

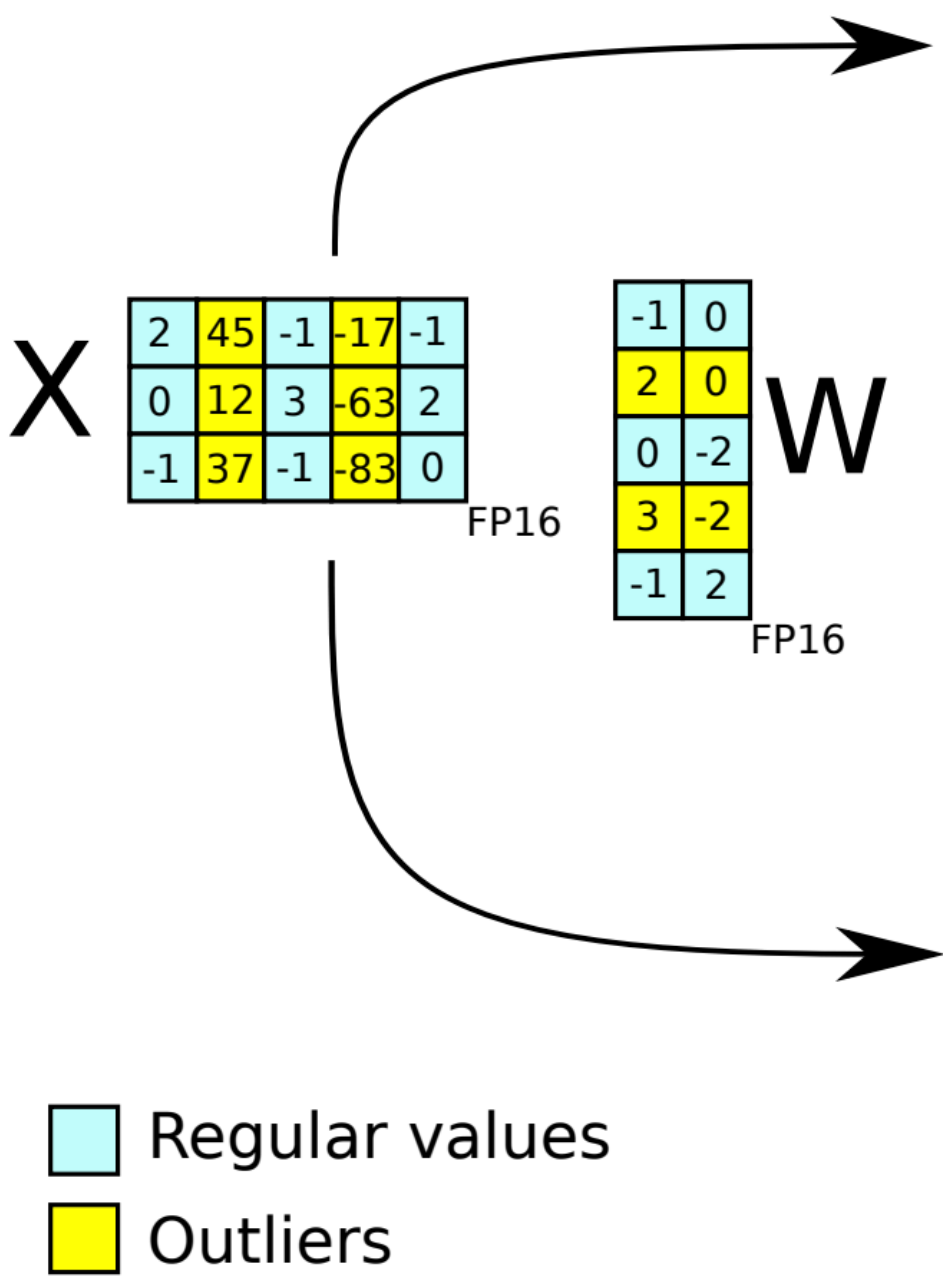
Динамическая квантизация. Используется в Hugging Face (bitsandbytes).



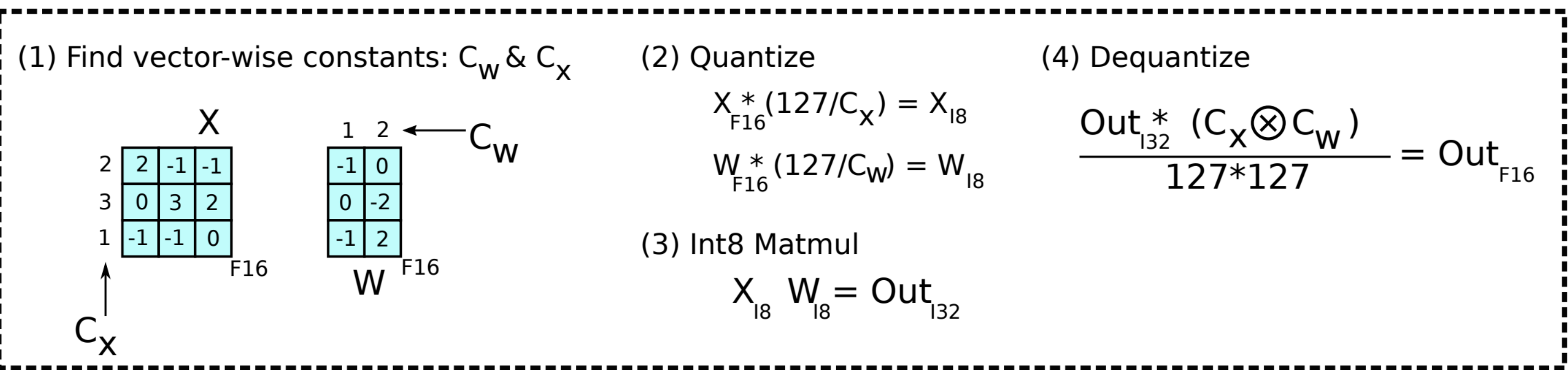
LLM.int8()

Динамическая квантизация. Используется в Hugging Face (bitsandbytes).

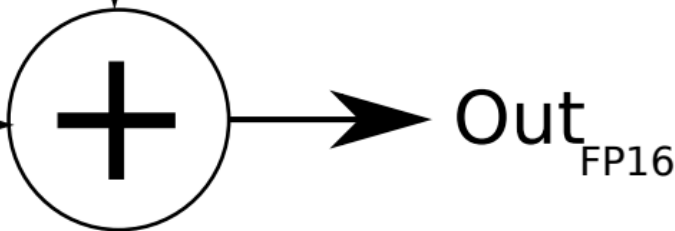
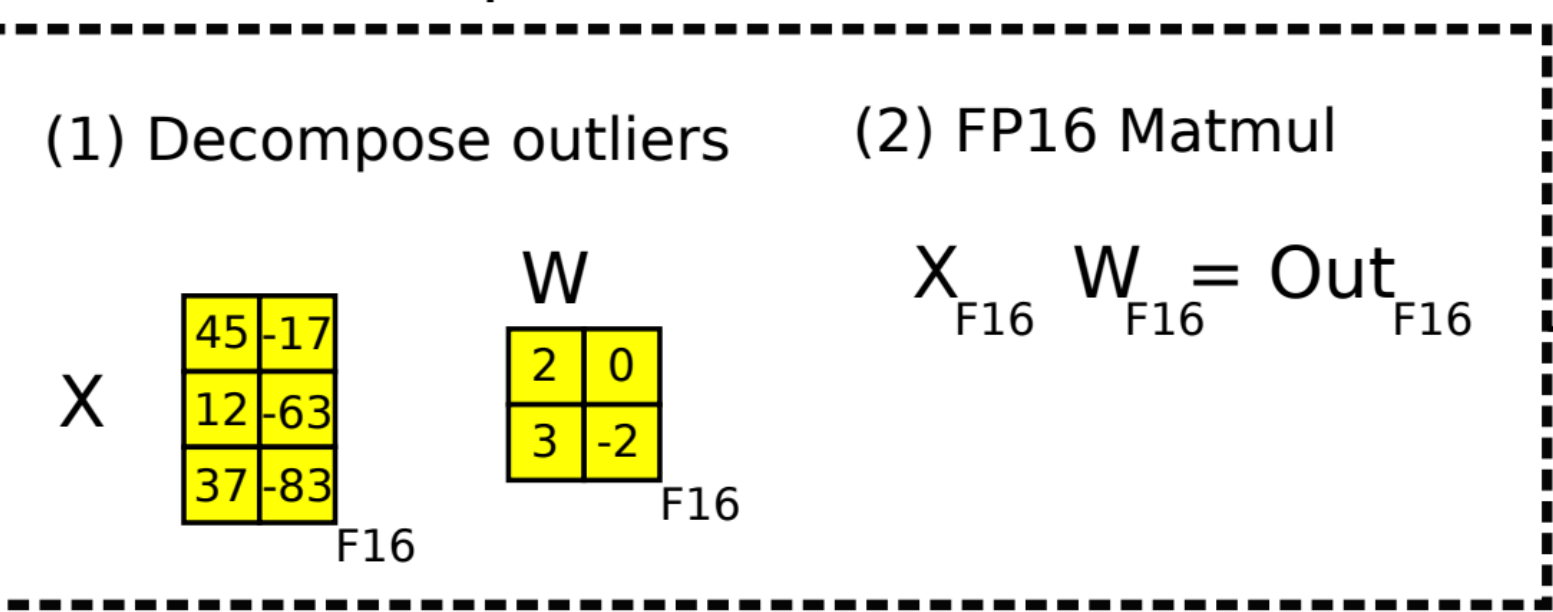
LLM.int8()



8-bit Vector-wise Quantization



16-bit Decomposition



LLM.int8()

Parameters	125M	1.3B	2.7B	6.7B	13B
32-bit Float	25.65	15.91	14.43	13.30	12.45
Int8 absmax	87.76	16.55	15.11	14.59	19.08
Int8 zeropoint	56.66	16.24	14.76	13.49	13.94
Int8 absmax row-wise	30.93	17.08	15.24	14.13	16.49
Int8 absmax vector-wise	35.84	16.82	14.98	14.13	16.48
Int8 zeropoint vector-wise	25.72	15.94	14.36	13.38	13.47
Int8 absmax row-wise + decomposition	30.76	16.19	14.65	13.25	12.46
Absmax LLM.int8() (vector-wise + decomp)	25.83	15.93	14.44	13.24	12.45
Zeropoint LLM.int8() (vector-wise + decomp)	25.69	15.92	14.43	13.24	12.45

Валидационная перплексия для моделей разных размеров.

GPTQ

aka Group-wise Precision Tuning Quantization
aka Generative Pre-trained Transformer Quantization

- Статическая квантизация. Позволяет сжимать веса до 3 бит.
- Идея основана на поочередной квантизации весов с корректировкой неквантизованных весов

$$w_q = \operatorname{argmin}_{w_q} \frac{(\operatorname{quant}(w_q) - w_q)^2}{[\mathbf{H}_F^{-1}]_{qq}}, \quad \delta_F = -\frac{w_q - \operatorname{quant}(w_q)}{[\mathbf{H}_F^{-1}]_{qq}} \cdot (\mathbf{H}_F^{-1})_{:,q}$$

- Работает быстро за счет оптимизаций подсчета гессиана

GPTQ

aka Group-wise Precision Tuning Quantization
aka Generative Pre-trained Transformer Quantization

Method	Bits	OPT-175B				BLOOM-176B			
		Wiki2	PTB	C4	LAMB. $\uparrow$	Wiki2	PTB	C4	LAMB. $\uparrow$
Baseline	16	8.34	12.01	10.13	75.59	8.11	14.59	11.71	67.40
RTN	4	10.54	14.22	11.61	71.34	8.37	15.00	12.04	66.70
GPTQ	4	8.37	12.26	10.28	76.80	8.21	14.75	11.81	67.71
RTN	3	7.3e3	8.0e3	4.6e3	0	571.	107.	598.	0.17
GPTQ	3	8.68	12.68	10.67	76.19	8.64	15.57	12.27	65.10
GPTQ	3/g1024	8.45	12.48	10.47	77.39	8.35	15.01	11.98	67.47
GPTQ	3/g128	8.45	12.37	10.36	76.42	8.26	14.89	11.85	67.86

Table 5: Results summary for OPT-175B and BLOOM-176B. “g1024” and “g128” denote results with groupings of size 1024 and 128, respectively.

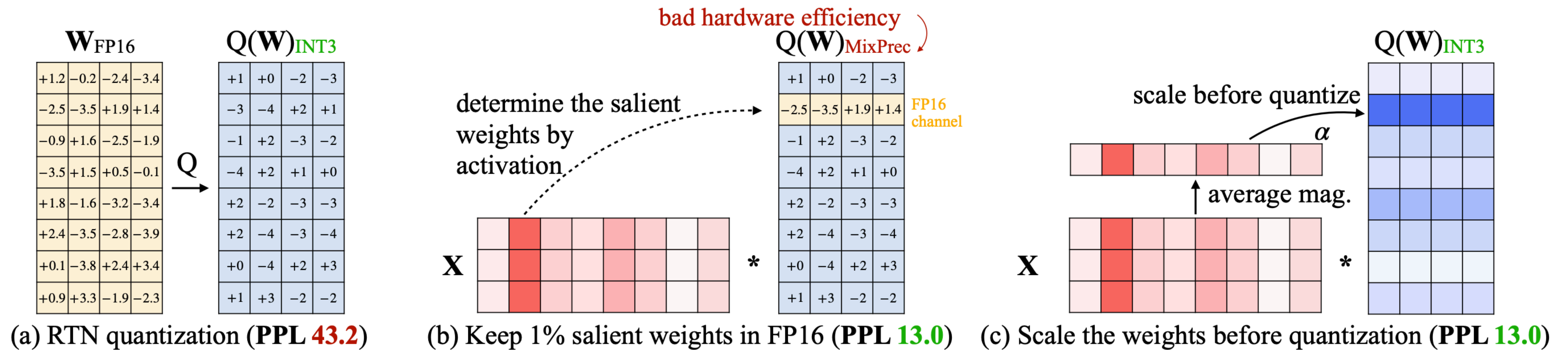
GPU	FP16	3bit	Speedup	GPU reduction
A6000 – 48GB	589ms	130ms	4.53 $\times$	8 $\rightarrow$ 2
A100 – 80GB	230ms	71ms	3.24 $\times$	5 $\rightarrow$ 1

AWQ

Activation-aware Weight Quantization

- Статическая квантизация.
- Вместо обработки выбросов в fp16, AWQ их масштабирует перед квантизацией

$$Q(w \cdot s) \cdot \frac{x}{s} = \Delta' \cdot \text{Round}\left(\frac{ws}{\Delta'}\right) \cdot x \cdot \frac{1}{s}$$



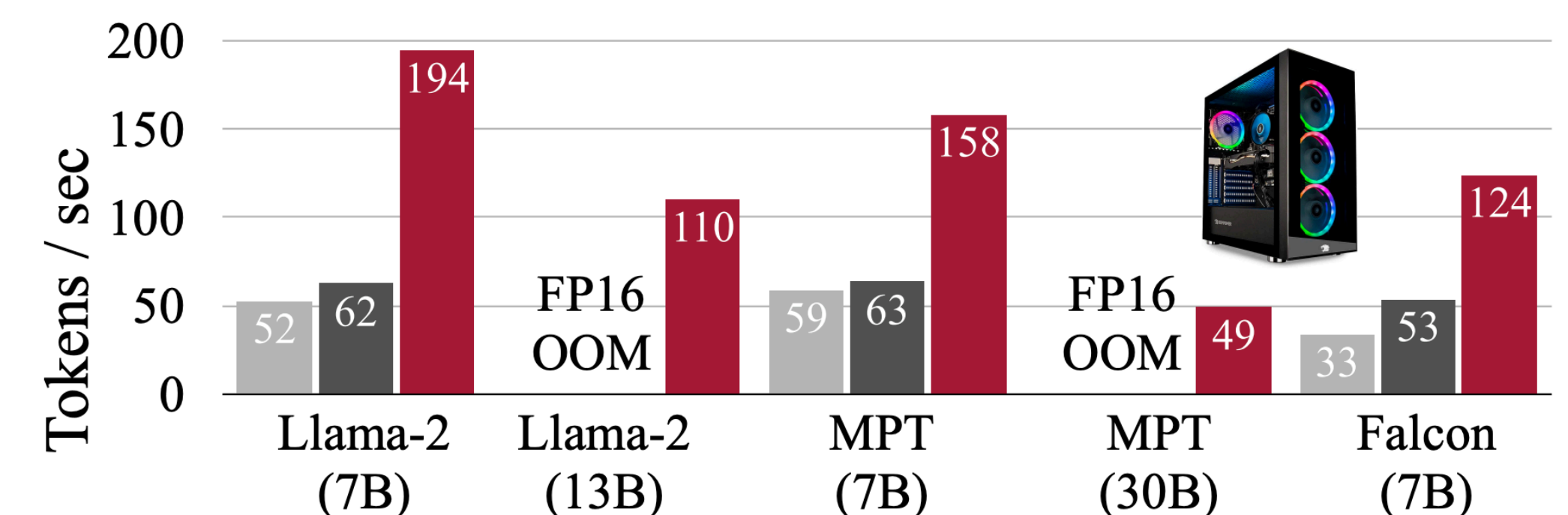
AWQ

Activation-aware Weight Quantization

PPL↓		Llama-2			LLaMA			
		7B	13B	70B	7B	13B	30B	65B
FP16	-	5.47	4.88	3.32	5.68	5.09	4.10	3.53
INT3 g128	RTN	6.66	5.52	3.98	7.01	5.88	4.88	4.24
	GPTQ	6.43	5.48	3.88	8.81	5.66	4.88	4.17
	GPTQ-R	6.42	5.41	3.86	6.53	5.64	4.74	4.21
	AWQ	6.24	5.32	3.74	6.35	5.52	4.61	3.95
INT4 g128	RTN	5.73	4.98	3.46	5.96	5.25	4.23	3.67
	GPTQ	5.69	4.98	3.42	6.22	5.23	4.24	3.66
	GPTQ-R	5.63	4.99	3.43	5.83	5.20	4.22	3.66
	AWQ	5.60	4.97	3.41	5.78	5.19	4.21	3.62

Huggingface (FP16)
 Ours (FP16)
 Ours (AWQ, W4A16)

- AWQ требует в 10 раз меньше данных для калибровки чем GPTQ
- Ускоряет инференс почти в 4 раза



(a) RTX 4090 desktop GPU

Спекулятивный декодинг

Мотивация

Скорость генерации значительно ограничена буттлнеком по памяти

На каждом шаге надо

- Перекладывать KV-кэш
- Загружать веса слоев в память

Пример

NVIDIA A100

Bandwidth: 2.04 TB/s

FP32: 19.5 TFLOPS

bs=1, seq_len=1000, dim=1000

Memory:

Weights: $O(dim \times dim \times 32)$ bytes

KV-cache: $O(seq_len \times dim \times n_layers \times 32)$ bytes

Time: **0.000384 s**

Compute:

$O(dim^2 + seq_len \times dim)$

Time: **0.00000005 s**

Спекулятивный декодинг

Идея:

Хотим декодировать токены параллельно, чтобы меньше гонять память за токен.

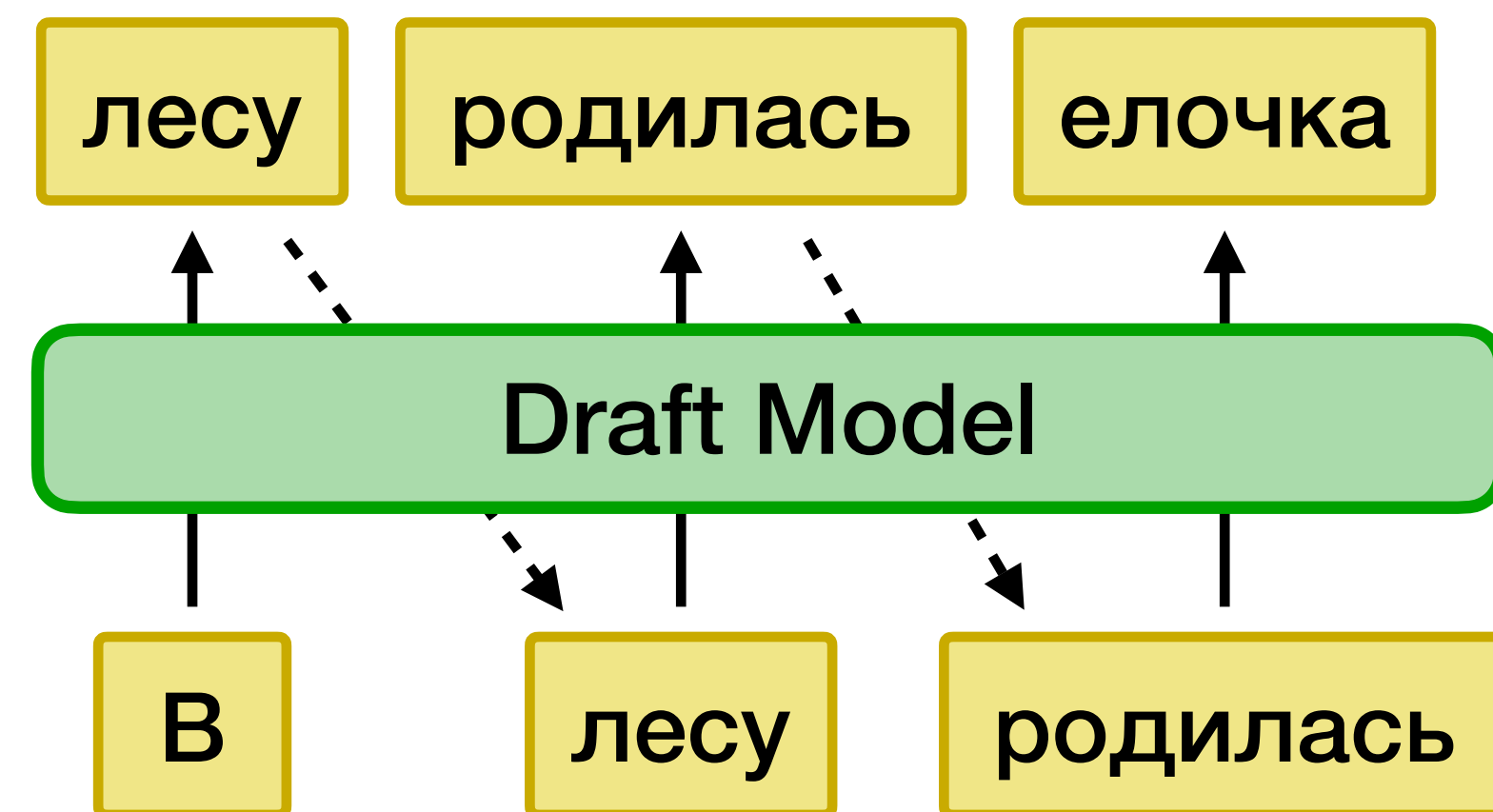
Проблема:

Авторегрессионная генерация разрешает генерировать только по одному токену за раз

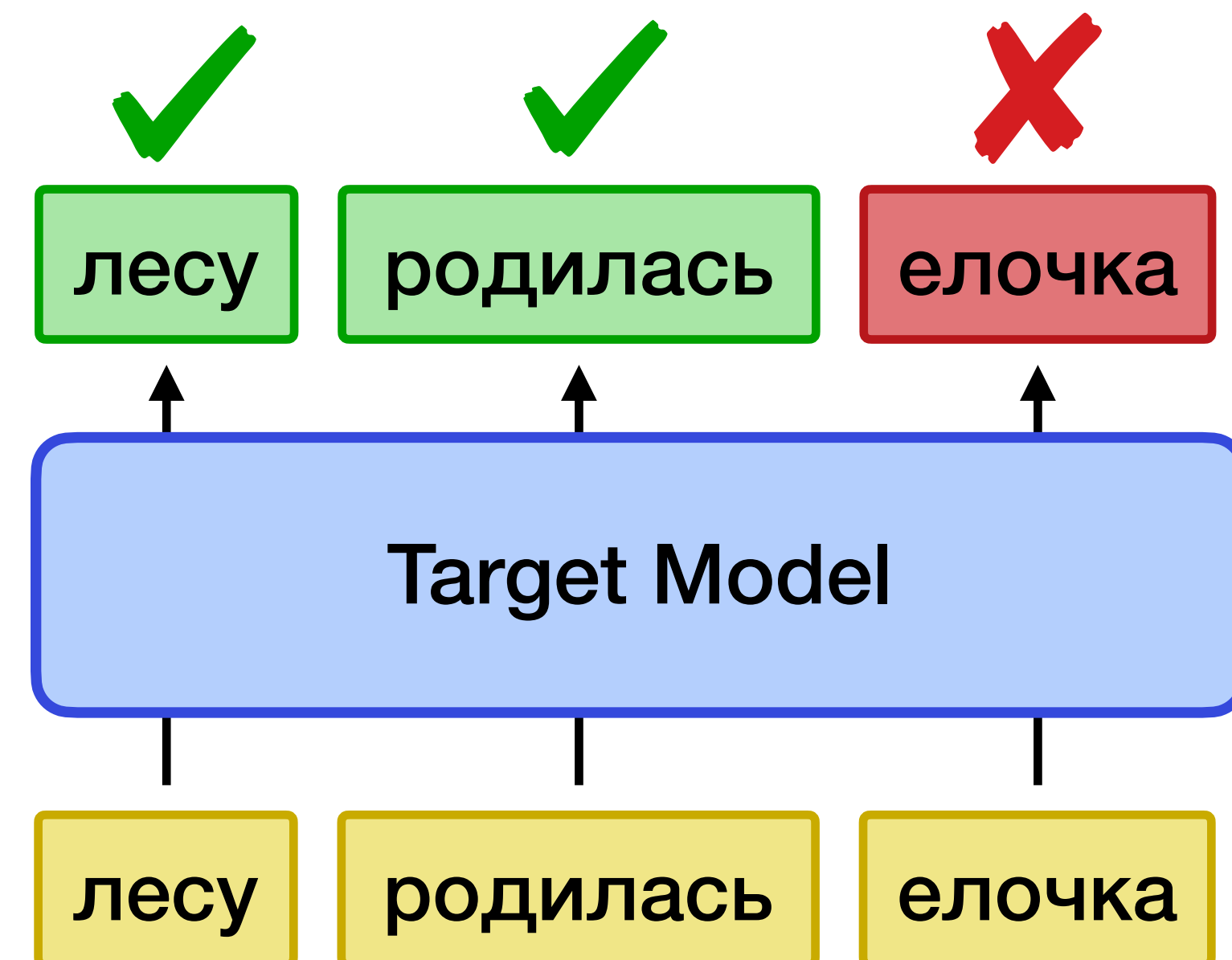
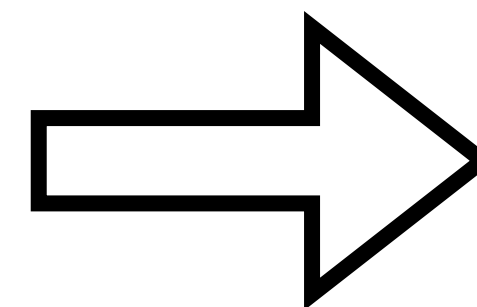
Решение:

Генерировать несколько токенов маленькой моделью, а большой моделью **параллельно** их проверять

Спекулятивный декодинг



Авторегрессионная генерация



Параллельная верификация

Алгоритм

1. Семплируем γ токенов драфтером
2. Параллельно считаем вероятности токенов верификатором
3. Принимаем токен с вероятностью
$$\min\left(1, \frac{p_i(x)}{q_i(x)}\right)$$
5. Первый отвергнутый токен пересемплируем из распределения

$$p'(x) = \text{norm}((p(x) - q(x))_+)$$

Algorithm 1 SpeculativeDecodingStep

Inputs: $M_p, M_q, prefix$.

▷ **Sample γ guesses $x_1, \dots, x_\gamma$ from M_q autoregressively.**

for $i = 1$ **to** γ **do**

$q_i(x) \leftarrow M_q(prefix + [x_1, \dots, x_{i-1}])$

$x_i \sim q_i(x)$

end for

▷ **Run M_p in parallel.**

$p_1(x), \dots, p_{\gamma+1}(x) \leftarrow$

$M_p(prefix), \dots, M_p(prefix + [x_1, \dots, x_\gamma])$

▷ **Determine the number of accepted guesses n .**

$r_1 \sim U(0, 1), \dots, r_\gamma \sim U(0, 1)$

$n \leftarrow \min(\{i - 1 \mid 1 \leq i \leq \gamma, r_i > \frac{p_i(x)}{q_i(x)}\} \cup \{\gamma\})$

▷ **Adjust the distribution from M_p if needed.**

$p'(x) \leftarrow p_{n+1}(x)$

if $n < \gamma$ **then**

$p'(x) \leftarrow \text{norm}(\max(0, p_{n+1}(x) - q_{n+1}(x)))$

end if

▷ **Return one token from M_p , and n tokens from M_q .**

$t \sim p'(x)$

return $prefix + [x_1, \dots, x_n, t]$

Сравнение с Rejection sampling

Rejection sampling

1. Sample $x \sim q(x)$ and $r \sim U(0, 1)$.
2. If $r < \frac{p(x)}{Mq(x)}$ return x .
3. Go to 1.

$$M = \max_x \frac{p(x)}{q(x)}$$

- Rejection sampling можно применять, однако его вероятность принятия токена будет значительно ниже.

Algorithm 1 SpeculativeDecodingStep

Inputs: $M_p, M_q, prefix$.

▷ **Sample γ guesses $x_1, \dots, x_\gamma$ from M_q autoregressively.**

for $i = 1$ **to** γ **do**

$q_i(x) \leftarrow M_q(prefix + [x_1, \dots, x_{i-1}])$

$x_i \sim q_i(x)$

end for

▷ **Run M_p in parallel.**

$p_1(x), \dots, p_{\gamma+1}(x) \leftarrow$

$M_p(prefix), \dots, M_p(prefix + [x_1, \dots, x_\gamma])$

▷ **Determine the number of accepted guesses n .**

$r_1 \sim U(0, 1), \dots, r_\gamma \sim U(0, 1)$

$n \leftarrow \min(\{i - 1 \mid 1 \leq i \leq \gamma, r_i > \frac{p_i(x)}{q_i(x)}\} \cup \{\gamma\})$

▷ **Adjust the distribution from M_p if needed.**

$p'(x) \leftarrow p_{n+1}(x)$

if $n < \gamma$ **then**

$p'(x) \leftarrow \text{norm}(\max(0, p_{n+1}(x) - q_{n+1}(x)))$

end if

▷ **Return one token from M_p , and n tokens from M_q .**

$t \sim p'(x)$

return $prefix + [x_1, \dots, x_n, t]$

Доказательство корректности

Пусть β – общая вероятность принятия следующего токена

$$\beta = \sum_x q(x) \min \left(1, \frac{p(x)}{q(x)} \right) = \sum_x \min(q(x), p(x))$$

Тогда заметим, что

$$p'(x) = \text{norm}(\max(0, p(x) - q(x))) = \frac{p(x) - \min(q(x), p(x))}{\sum_{x'} p(x') - \min(q(x'), p(x'))} = \frac{p(x) - \min(q(x), p(x))}{1 - \beta}$$

Теперь

$$P(x = x') = P(\text{guess accepted}, x = x') + P(\text{guess rejected}, x = x')$$

$$P(\text{guess accepted}, x = x') = q(x') \min \left(1, \frac{p(x')}{q(x')} \right) = \min(q(x'), p(x'))$$

$$P(\text{guess rejected}, x = x') = (1 - \beta)p'(x') = p(x') - \min(q(x'), p(x'))$$

Следовательно

$$P(x = x') = \min(q(x'), p(x')) + p(x') - \min(q(x'), p(x')) = p(x')$$